# Kirana: A Multimodal Grocery Finance Agent

Personalised household receipt QA over visual and structured data

## 1. Introduction

Students who share groceries with housemates face a fragmented financial picture. The typical pattern is that one person pays the full bill at the register, sends a split request on Splitwise, and then roommates trade counter-splits back and forth over weeks : none of which are ever cleanly settled. A student might pay $120 at Woolworths one week and $20 two weeks later as a counter-split, with no easy way to reconcile these flows against actual receipts. The result is that your true grocery spend in any given month is buried across dozens of unresolved Splitwise entries and receipt images you no longer remember.

Beyond reconciliation, there is a price intelligence dimension: a student asking "**how much do I usually pay for wraps?"** or "**which store has cheaper onions**?" needs to query across months of receipt line items, requiring both exact structured lookup and semantic flexibility over how item names vary across receipts and stores. Kirana is designed to be a single conversational interface for any question a student might have about their grocery purchases : what they bought, what it cost, who paid, and whether they are getting a fair price.

## 2. Knowledge Base Construction

The knowledge base is built from two personalised sources ingested independently.

**Receipt images.** Real household grocery receipts photographed from supermarkets (Woolworths, Coles) and local stores across Brisbane. Each image is a primary evidence artefact.

**Splitwise CSV export.** A manual export of the shared household expense ledger, providing per-person cost contributions independent of receipt OCR.

This yields two modalities: image (receipt photographs) and structured text (Splitwise CSV rows and VLM-extracted receipt JSON). The two sources are intentionally kept separate : receipts provide item-level evidence; Splitwise provides accounting truth for personal shares. The join between them is intentionally deferred to query time because Splitwise dates and amounts may drift by one or two days and one or two percent from receipt totals.

## 3. Retrieval and Representation Design

### 3.1 Ingestion pipeline

Each receipt image passes through a five-stage pipeline. Gemini 3.1 Flash lite preview (via Cloudflare AI Gateway) extracts a structured JSON document containing store name, date, address, line items with quantities and prices, subtotal, GST, and discounts. A Pydantic validator checks field completeness and accounting consistency; failures trigger a retry with the error injected into the prompt. After validation, Mistral generates a short natural-language summary capturing the semantic character of the shopping trip (category skew, notable items, scale) rather than restating every line item. The summary is embedded locally with Qwen3-VL-Embedding-2B and upserted into Cloudflare Vectorize. Structured rows are written to Cloudflare D1. Raw images and derived artefacts are archived in Tigris object storage.

Splitwise CSV rows are imported directly into Cloudflare D1 (RDBMS) without VLM or embedding steps.

### 3.2 Dual vector index design

Two separate Cloudflare Vectorize indexes support different retrieval surfaces. The receipt-summary index (kirana-receipt-summaries) stores one vector per receipt, embedding the Mistral-generated semantic summary with metadata for receipt ID, store name, and date. The item-name index (kirana-receipt-items) stores one vector per distinct item name drawn from all ingested line items. This separation is deliberate: trip-level semantic search ("find snack-heavy trips in February") queries the summary index, while query-time image matching queries the item-name index using an image embedding produced at request time. Mixing the two retrieval surfaces into one index would conflate trip-level and item-level similarity. Splitwise data is stored in Cloudflare D1 (RDBMS) and exposed through computed SQL views : me_expenses, roommate1_expenses, and roommate2_expenses : that convert the raw accounting-style CSV rows (which record who paid and raw per-person amounts) into LLM-queryable form with an adjusted my_share_cents column. This means the agent can answer "what was my personal grocery contribution in March?" with a single aggregation over me_expenses rather than reasoning over raw split arithmetic.

**Query-time multimodality.** Uploaded item images are embedded at query time using the same Qwen3-VL-Embedding-2B model. The resulting vector is matched against the item-name index; top candidates are then confirmed through SQL joins over receipt_items and receipts. This means the system performs cross-modal retrieval without requiring images to be stored at ingestion time.

## 4. Agent Workflow

Kirana uses a LangGraph ReAct like agent loop with four tools: query_finance_db (read transaction data from RDBMS), embed_receipt_query, search_receipt_vectors (searches the receipt-summary Cloudflare Vectorize index with optional metadata filters like date), and search_item_vectors (searches the item-name Cloudflare Vectorize index, used for image questions).

The agent decides tool sequences based on query intent. Factual aggregation queries (QF1) and conversational drill-downs (QF4) resolve entirely through SQL. Semantic trip queries (QF3) use embed → vector search → SQL confirmation in sequence. Uploaded image queries (QF2) skip embedding and call search_item_vectors directly with the pre-computed image embedding key, then confirm matches through SQL.

A plain retrieval pipeline cannot serve all four families with a fixed sequence: SQL alone has no path for image or semantic queries, and vector search alone cannot perform date-window aggregation or Splitwise reconciliation. The agent layer is the minimal mechanism that routes between these heterogeneous retrieval paths based on the query.

Splitwise reconciliation is handled at query time using a ±1 day date window and the me_expenses adjusted-share view, rather than joining receipt rows to split rows on date. The agent is instructed to retrieve household totals and personal shares as independent queries and reconcile in reasoning, handling cases where one Splitwise entry spans multiple receipts.

## 5. Experimental Setup

Three conditions are compared across four query families and 17 test cases.

We ablate over system prompt engineering strategy rather than retrieval architecture because the prompt is the primary lever a practitioner controls post-deployment. The knowledge base, embedding model, Cloudflare Vectorize indexes, and Cloudflare D1 schema are fixed design decisions; the system prompt is what a developer iterates on. In-context learning (ICL) is widely seen as a lower-maintenance alternative to hand-written procedural instructions: rather than anticipating every edge case as an explicit rule, you demonstrate correct behaviour through examples and let the model generalise. For a generic QA agent this trade-off is often reasonable. Kirana, however, operates over a domain with non-obvious constraints: Splitwise entries drift by one or two days from receipt dates, personal shares come from a computed view rather than raw split rows, and image recall requires gating on a similarity threshold before answering. These constraints are unlikely to be inferred from a single demonstration per query family. The ablation therefore tests a practically motivated hypothesis: **for tool-calling agents with domain-specific financial backends, does explicit rule encoding in the system prompt outperform sparse demonstrations, particularly on query families where the correct tool sequence is non-obvious?**

**Conditions.**

1. (Plain LLM) The underlying model with no tools and no knowledge base access. The model receives only the user query and has no visibility into receipts, items, or Splitwise data, making it a true zero-knowledge baseline.
2. (A0) The full agent with a procedural system prompt: a detailed set of explicit rules covering source selection (when to use receipts vs me_expenses), Splitwise reconciliation procedure (query independently with a ±1 day window, never join receipts to splits on date), date range handling, image query routing, and similarity threshold gating.
3. (A1) The same agent with a sparse in-context learning (ICL) prompt that replaces all explicit rules with one worked example per query family : showing a user query, the correct tool call sequence, an example tool result, and the final answer : and no procedural instructions. A1 tests whether demonstrations alone are sufficient to teach the domain-specific constraints that A0 encodes as rules.

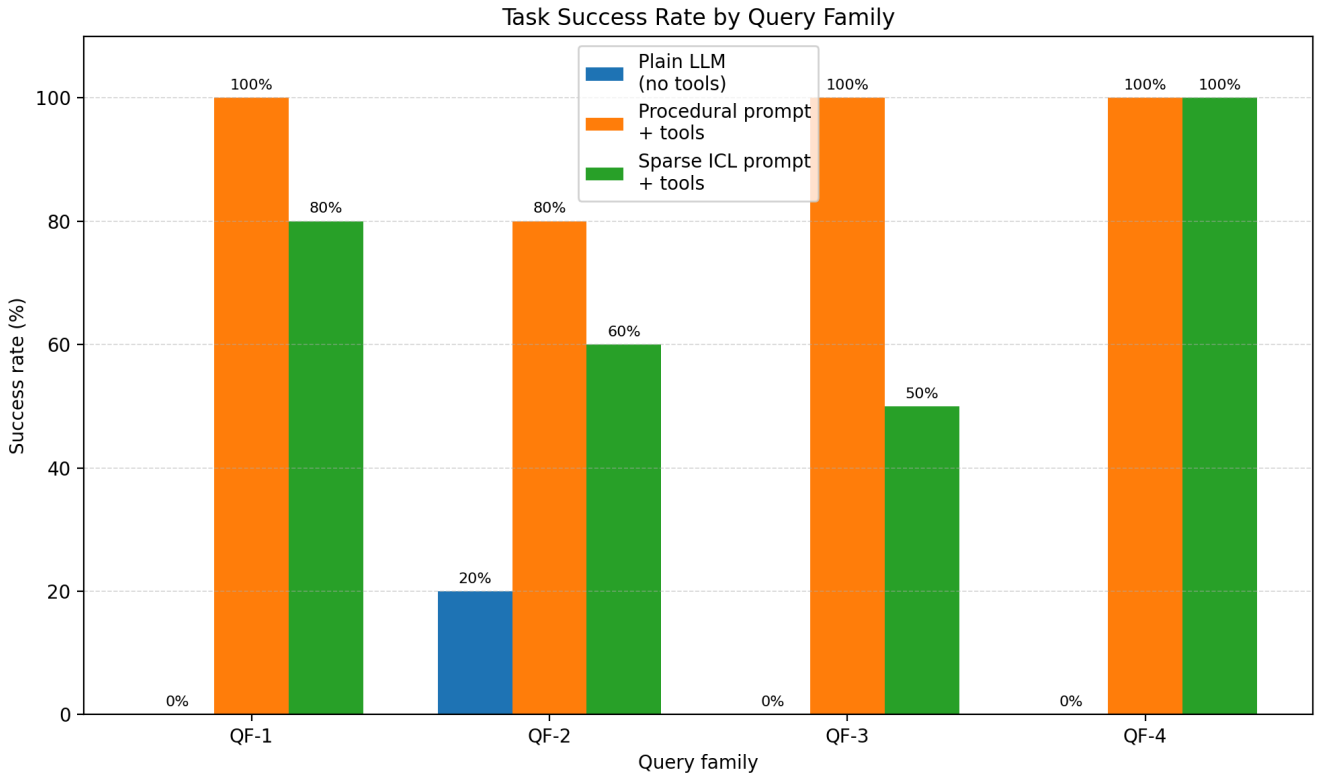
**Metrics.** Task success rate (binary pass/fail; numerical answers accept ±$3 or ±5%). LLM-as-judge groundedness for QF3. Token consumption per test case as the efficiency metric (latency excluded due to Cloudflare AI Gateway prompt-cache effects that produced non-comparable measurements across conditions).

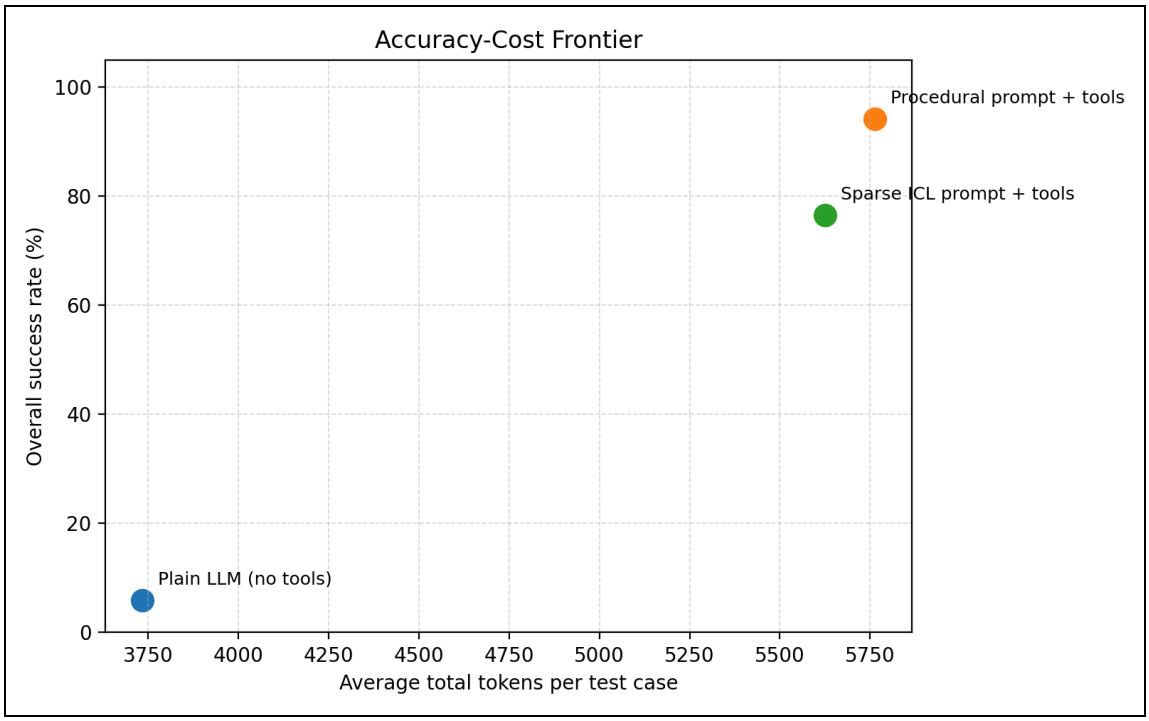
## 6. Benchmark Results and Analysis

| Condition | QF-1 | QF-2 | QF-3 | QF-4 |
|---|---|---|---|---|
| BASELINE | | | | |
| Plain LLM (no tools) | 0/5 | 1/5 | 0/2 | 0/5 |
| ABLATION | | | | |
| Procedural prompt + tools | 5/5 | 4/5 | 2/2 | 5/5 |
| ↳ sparse ICL prompt + tools | 4/5 | 3/5 | 1/2 | 5/5 |
| OVERALL TASK SUCCESS RATE | | | | |
| Plain LLM (no tools) | 6% (1/17) | | | |
| Procedural prompt + tools | 94% (16/17) | | | |
| ↳ sparse ICL prompt + tools | 76% (13/17) | | | |

The full agent (A0) achieves 94% overall task success against 6% for the plain LLM baseline, confirming that agentic tool use over the structured knowledge base is the primary driver of performance. The baseline has no access to receipts, items, or Splitwise data and fails across all four query families. Its single passing case (QF2, 1/5) is a coincidental correct answer produced without any grounding evidence.

The ablation (A0 vs A1) shows that sparse ICL degrades on domain-constrained families. QF3 drops from 100% to 50%: without explicit reconciliation rules, the sparse ICL agent reverts to naive same-date joins between receipts and splits, producing incorrect personal share values. QF2 drops from 80% to 60%: without threshold guidance or a negative-match example, the agent returns false positives on low-similarity image queries. QF1 drops from 100% to 80% due to occasional incorrect month boundary handling. QF4 is unaffected (100%), consistent with conversational carry-forward being learnable from a single demonstration.






On the accuracy-cost frontier, A0 dominates A1: it achieves higher accuracy at comparable token cost (~5,750 vs ~5,550 tokens per test case). Sparse ICL reduces token consumption by only 3.5% while dropping overall accuracy by 18 percentage points, offering no Pareto improvement over the procedural baseline. This result suggests that for tool-calling agents operating over domain-specific backends with non-obvious constraints (fuzzy date alignment, adjusted expense views, similarity thresholds), explicit constraint encoding in the system prompt outperforms demonstration-based prompting.

## 7. Failure Cases

**QF2 false positives (A1).** Without an explicit similarity threshold rule in the system prompt, the A1 agent does not gate on the Cloudflare Vectorize match score before answering. When a low-confidence image query returns candidates scoring below the configured threshold, A1 treats them as valid matches and answers Yes. A0 encodes the threshold gate as an explicit instruction and correctly answers No. The failure is in prompt-guided orchestration, not in retrieval: Vectorize surfaces the same low-scoring candidates in both conditions, but only A0 knows to reject them.

**QF3 reconciliation errors (A1).** The A0 procedural prompt explicitly prohibits joining the receipts table to splits on date and requires the agent to query me_expenses independently over a ±1 day window. A1 has no such rule. When asked for a personal contribution breakdown, the A1 agent instead constructs a SQL query that joins receipts to splits on exact date. Because Splitwise entries frequently land one day after the receipt date (the split is sent after shopping, not during), the join returns no rows and the agent incorrectly reports that no personal contribution was recorded for that shopping trip. The retrieval infrastructure (Cloudflare D1, the me_expenses view) is identical in both conditions; the failure is entirely in how the agent is instructed to use it.

**Plain LLM hallucination (all families).** Without tool access, the model fabricates plausible-sounding totals from the injected context sample, occasionally matching on QF2 by coincidence. No QF1 or QF4 case passes because the model invents arithmetic that does not correspond to the actual receipt dataset.

## 8. Conclusion

Kirana demonstrates that a multimodal grocery finance agent can reliably answer household receipt queries that neither plain LLMs nor SQL systems alone can handle. The dual-index retrieval architecture, VLM ingestion pipeline, and query-time image matching each address a distinct gap in the retrieval surface. The ablation reveals that for agents operating over structured financial backends with domain-specific constraints, procedural system prompts encoding explicit rules substantially outperform sparse in-context demonstrations : a finding that generalises to any tool-calling agent where the correct tool sequence is non-obvious and domain rules are precise.